



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Robótica Móvil

Labs 1-I, 1-II, 2, 3, 4, 5 y 6

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: ROS 2 Humble · CoppeliaSim · MATLAB

Índice general

1. Lab-MR 1-I: Seguimiento de caminos explícitos

1.1. Objetivo

Completar el método `controlLoop()` del nodo `nav_p2p.cpp` (paquete `seg_tray`) para que el robot MATLAB simulado en CoppeliaSim recorra una secuencia de waypoints definida en `nav_params.yaml` mediante navegación punto a punto con control proporcional de rumbo. Una vez verificado el recorrido completo, modificar la ganancia K y comentar cómo influye en la trayectoria del robot.

1.2. Implementación

El nodo `nav_p2p_node` ya proporcionado en `seg_tray` se encarga de cargar los waypoints del YAML, leer la odometría y publicar comandos de velocidad a 40 Hz. Lo único que hay que completar es el bloque indicado dentro de `controlLoop()` con las ecuaciones del enunciado. Las decisiones no obvias son:

Cambio a coordenadas locales del vehículo. El error de orientación ϕ_e se obtiene más directamente expresando el waypoint en el sistema de referencia del robot. Aplicando la rotación del enunciado:

$$x_{P_l} = \cos \phi_o (x_P - x) + \sin \phi_o (y_P - y), \quad y_{P_l} = -\sin \phi_o (x_P - x) + \cos \phi_o (y_P - y),$$

queda $\phi_e = \text{atan2}(y_{P_l}, x_{P_l})$, que ya está en el rango correcto sin necesidad de normalizaciones explícitas.

Control proporcional y cinemática inversa. La curvatura comandada es $\gamma = K \phi_e$ y las velocidades angulares de cada rueda se obtienen aplicando directamente las ecuaciones del enunciado:

$$\omega_i = \frac{v(1 - K \gamma)}{R}, \quad \omega_d = \frac{v(1 + K \gamma)}{R}.$$

A partir de (ω_i, ω_d) se recuperan las velocidades del vehículo (v, ω) con la cinemática directa y se publican en `cmd_vel`.

Cambio de waypoint a 1 m. El waypoint se considera alcanzado cuando la distancia euclídea al punto objetivo es inferior a 1 m:

$$\sqrt{(x_P - x)^2 + (y_P - y)^2} < 1 \text{ m},$$

tal y como indica el enunciado, y se pasa al siguiente. Cuando no quedan waypoints, se publica $v = \omega = 0$.

1.3. Pruebas

Tras lanzar la escena `p2p_scene.ttt` en CoppeliaSim y ejecutar `ros2 launch seg_tray nav_p2p_launch.py`, el robot recorre los 7 waypoints del fichero de configuración a $v = 1,2 \text{ m s}^{-1}$ y completa el camino. La Figura ?? muestra el recorrido sobre el plano del simulador.

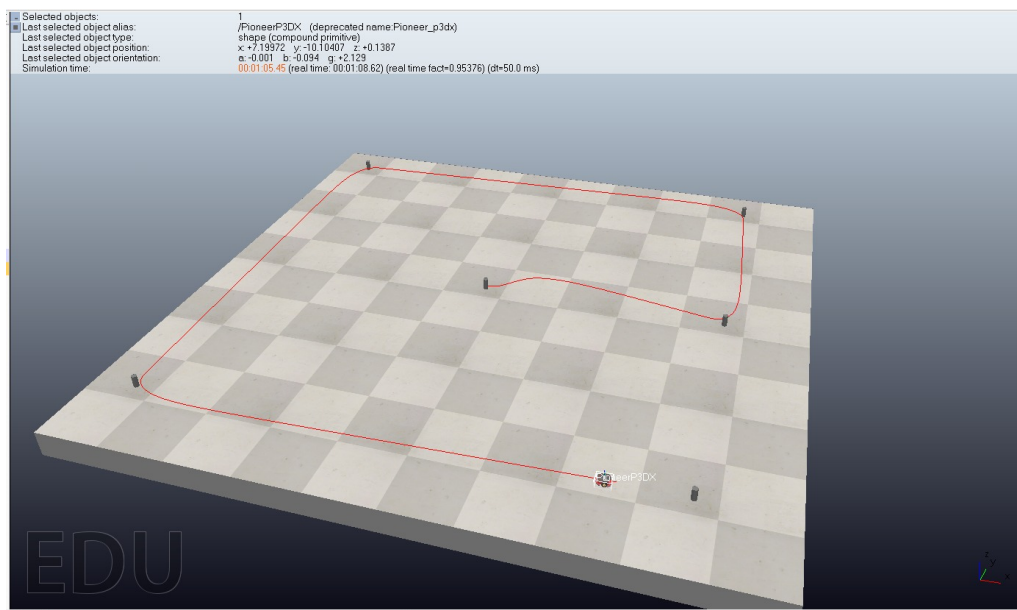


Figura 1.1: Recorrido rectangular del MATLAB con control P2P en CoppeliaSim.

1.3.1. Influencia de la ganancia K

El enunciado pide modificar la ganancia y comentar el efecto sobre la trayectoria:

- Con K **pequeño** (e.g. $K = 1$) el robot corrige el rumbo lentamente y se aleja lateralmente del waypoint antes de orientarse, dibujando arcos amplios entre puntos consecutivos.
- Con K **moderado** (e.g. $K = 4$) la respuesta es rápida y la trayectoria entre waypoints es prácticamente recta, con curvas suaves en los cambios de waypoint.
- Con K **grande** (e.g. $K \gtrsim 6$) la curvatura comandada se vuelve excesiva y aparecen oscilaciones porque el error angular cambia de signo de un ciclo al

siguiente.

El valor $K = 4$ ofrece un buen compromiso entre rapidez y estabilidad para este recorrido.

1.4. Comentarios finales

El controlador implementado cumple lo que pide el enunciado: el robot sigue la secuencia de waypoints con control proporcional sobre el error de orientación calculado en coordenadas locales, y la ganancia K actúa como esperado según el análisis del parámetro. La estructura del nodo proporcionado ($1,2 \text{ m s}^{-1}$, periodo 25 ms , tolerancia 1 m) es suficiente para completar el recorrido en el escenario sin obstáculos.

2. Lab-MR 1-II: Seguimiento de caminos implícitos

2.1. Objetivo

Completar el método `controlLoop()` del nodo `corr_nav.cpp` (paquete `seg_tray`) para que el robot MATLAB navegue por el centro del pasillo aplicando el método de **persecución pura** (Pure Pursuit) sobre un punto objetivo situado a una distancia de *look-ahead* $L = 1$ m por delante. El código proporcionado ya incluye la lectura del láser y la función `averageRangeInWindow` que estima la distancia a paredes y al fondo; lo que hay que implementar es la ley de control. Una vez funcionando, comprobar el efecto de distintas condiciones iniciales (posición y orientación del robot) sobre el seguimiento.

2.2. Implementación

El nodo `corr_nav_node` ya se suscribe a los topics de láser y odometría, ejecuta `averageRangeInWindow` en `laserCallback()` para obtener distancias laterales, frontal y de fondo, y publica velocidades a 40 Hz. En `controlLoop()` se completa la lógica con dos comportamientos. Las decisiones no obvias son:

Centrado por error lateral y orientación. Con d_{left} , d_{right} y la anchura de pasillo W (parámetro YAML), el error lateral es $e_{\text{lat}} = d_{\text{left}} - W/2$. La orientación del robot respecto al eje del pasillo se estima geoméricamente como $|\theta| = \arccos(W/(d_{\text{left}} + d_{\text{right}}))$, y su signo se determina comparando lecturas a $\pm 15^\circ$ de cada perpendicular: si la lectura “delantera” es menor que la “trasera”, el robot apunta hacia esa pared.

Punto objetivo en el centro del pasillo y curvatura de Pure Pursuit. La coordenada lateral del punto de look-ahead en el sistema del robot es $y_L = e_{\text{lat}} \cos \theta - L \sin \theta$, y la curvatura comandada es $\kappa = 2y_L/L^2$. La velocidad lineal se fija a $0,25 \text{ m s}^{-1}$ (un cuarto de v_{max}) y la angular es $\omega = \kappa v$ saturada a $\pm \omega_{\text{max}}$. Con $L = 1$ m se obtiene un comportamiento estable sobre el ancho del pasillo configurado.

Modo curva activado por anticipación a -45° . El sector lateral perpendicular ($\pm 90^\circ$) sigue “viendo” la pared del tramo recto hasta el último instante, lo que provocaba colisiones al entrar en una curva. Para anticiparlas se añade una

lectura a -45° con componente frontal: si $d_{\text{right}} > W$ o $d_{\text{right_ahead}} > W$ se conmuta a un modo de giro con $v = 0,125 \text{ m s}^{-1}$ y $\omega = -\omega_{\text{max}}$ hasta volver a tener pared a la derecha. Sin esta anticipación el robot no negocia las curvas a tiempo.

Aprovechar el código proporcionado en lugar de reescribirlo. El enunciado entrega `averageRangeInWindow` y la estructura de callbacks. La implementación se limita a usar esas distancias y a añadir las pocas líneas que pide el método de persecución pura, sin tocar el código sensorial.

2.3. Pruebas

Tras lanzar la escena en CoppeliaSim y ejecutar el nodo de navegación, el robot recorre el pasillo manteniéndose centrado y negocia las curvas a la derecha gracias al modo de giro anticipado.

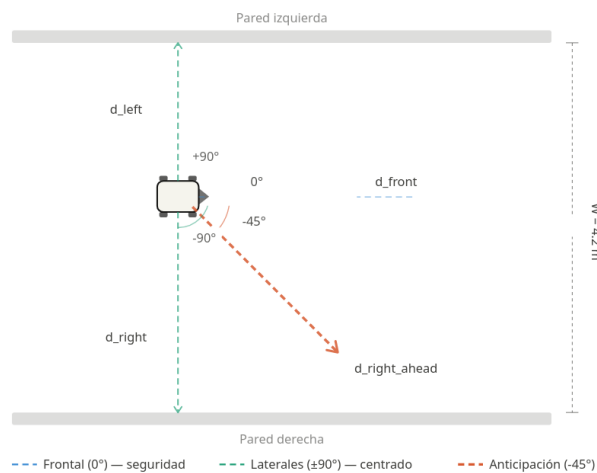


Figura 2.1: Disposición de los haces láser utilizados: laterales a $\pm 90^\circ$ para el centrado, frontal a 0° para seguridad, y anticipación a -45° para detección de curvas.

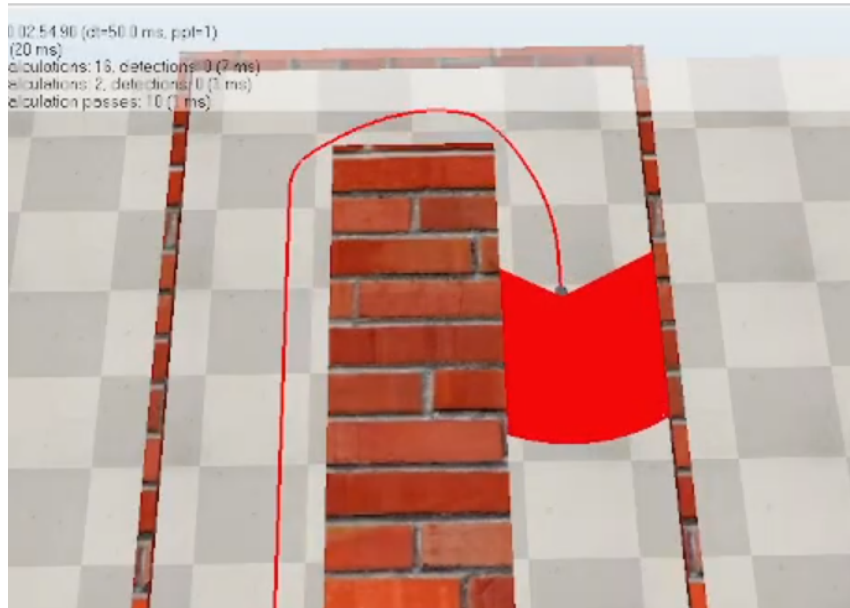


Figura 2.2: Robot MATLAB navegando por el pasillo en Coppeliasim: tramo recto centrado y transición al modo curva.

2.3.1. Efecto de las condiciones iniciales

Modificando con las herramientas de mover y girar de Coppeliasim la posición y la orientación del robot antes de iniciar la simulación se observa lo siguiente:

- Con el robot **descentrado** pero orientado en la dirección del pasillo, Pure Pursuit corrige el error lateral suavemente y el robot se centra en pocos metros sin oscilar.
- Con el robot **rotado** respecto al eje del pasillo (hasta $\pm 30^\circ$ aproximadamente), la estimación geométrica de θ funciona y el control reduce el error angular junto con el lateral; el centrado tarda algo más pero se alcanza.
- Con **rotaciones grandes** (cerca de $\pm 90^\circ$), una de las paredes deja de ser visible para los haces laterales, $|\theta|$ no se puede calcular con la fórmula del arccos y el comportamiento se degrada hasta que el robot vuelve a alinearse aproximadamente con el pasillo.

En todos los casos en los que ambas paredes son visibles, el método recupera el centrado: Pure Pursuit con $L = 1$ m es robusto a perturbaciones moderadas porque el punto objetivo se reconstruye en cada ciclo a partir del estado estimado.

2.4. Comentarios finales

La ley de persecución pura cumple lo que pide el enunciado: el robot navega por el centro del pasillo y soporta perturbaciones de las condiciones iniciales mientras ambas paredes sean visibles. La adición del modo curva con anticipación a -45° no la pide el enunciado explícitamente, pero es necesaria para que el seguimiento del pasillo no termine en colisión al llegar al primer giro. El servicio de aviso por proximidad a la pared del fondo (apartado 3 del enunciado) no se ha implementado en esta entrega.

3. Lab-MR 2: Percepción del entorno

3.1. Objetivo

Sustituir la función `averageRangeInWindow()` de Lab-MR 1-II por una función `extractWall()` que estime cada pared del pasillo como una recta $y = mx + b$ mediante **regresión lineal por mínimos cuadrados** sobre los puntos láser. Crear el nodo `corr_nav_wall` (basado en el de la práctica anterior), adaptar el bucle de control para usar las dos rectas en el cálculo del punto objetivo de Pure Pursuit, compilar y lanzar con `ros2 launch seg_tray corr_nav_wall_launch.py`.

3.2. Implementación

El nodo `corr_nav_wall_node` se ha derivado del de Lab-MR 1-II reemplazando `averageRangeInWindow()` por `extractWall()` y eliminando el servicio que ya no se necesita. La estructura general (callback de láser, callback de odometría, bucle de control a 40 Hz que publica `cmd_vel`) se mantiene. Las decisiones no obvias son:

Rangos angulares estrechos. La pared izquierda se extrae del sector $[60^\circ, 120^\circ]$ y la derecha del sector $[-120^\circ, -60^\circ]$. Rangos más amplios (e.g. $\pm 30^\circ$ a $\pm 150^\circ$) incluyen lecturas de las esquinas del pasillo, que sesgan la regresión y hacen oscilar la orientación estimada.

Filtrado de lecturas inválidas. Se descartan rangos no finitos o no positivos antes de convertir a cartesianas. Si quedan menos de 2 puntos válidos, la pared se marca como no detectada ($b = +\infty$).

Protección frente a paredes verticales. La fórmula de mínimos cuadrados para m tiene como denominador $n \sum x_i^2 - (\sum x_i)^2$, que tiende a cero cuando los puntos comparten la misma x (pared perpendicular al eje del robot). En ese caso se devuelve $m = 0$ y $b = \bar{y}$, lo que equivale a tratar la pared como horizontal y evita un NaN.

Punto objetivo analítico. Conocidas (m_L, b_L) y (m_R, b_R) , el centro del pasillo a la distancia de look-ahead L es directamente

$$y_{\text{goal}} = \frac{m_L + m_R}{2} L + \frac{b_L + b_R}{2},$$

y la curvatura de Pure Pursuit es $\kappa = 2 y_{\text{goal}} / L^2$. No hace falta promediar lecturas a izquierda y derecha como en Lab-MR 1-II: el centrado y la orientación quedan absorbidos por el ajuste lineal.

Cláusula de guarda al perder una pared. Si b_L o b_R no son finitos (e.g. en una curva el sector angular deja de ver una pared), se publica $v = \omega = 0$ en vez de calcular comandos erráticos. Es una protección de seguridad más que un modo de curva.

3.3. Pruebas

Tras compilar e incluir el ejecutable en `CMakeLists.txt` y crear el launch file copiado del de Lab-MR 1-II, el nodo se lanza y el robot navega por el centro del pasillo en el escenario de CoppeliaSim (Figura ??).

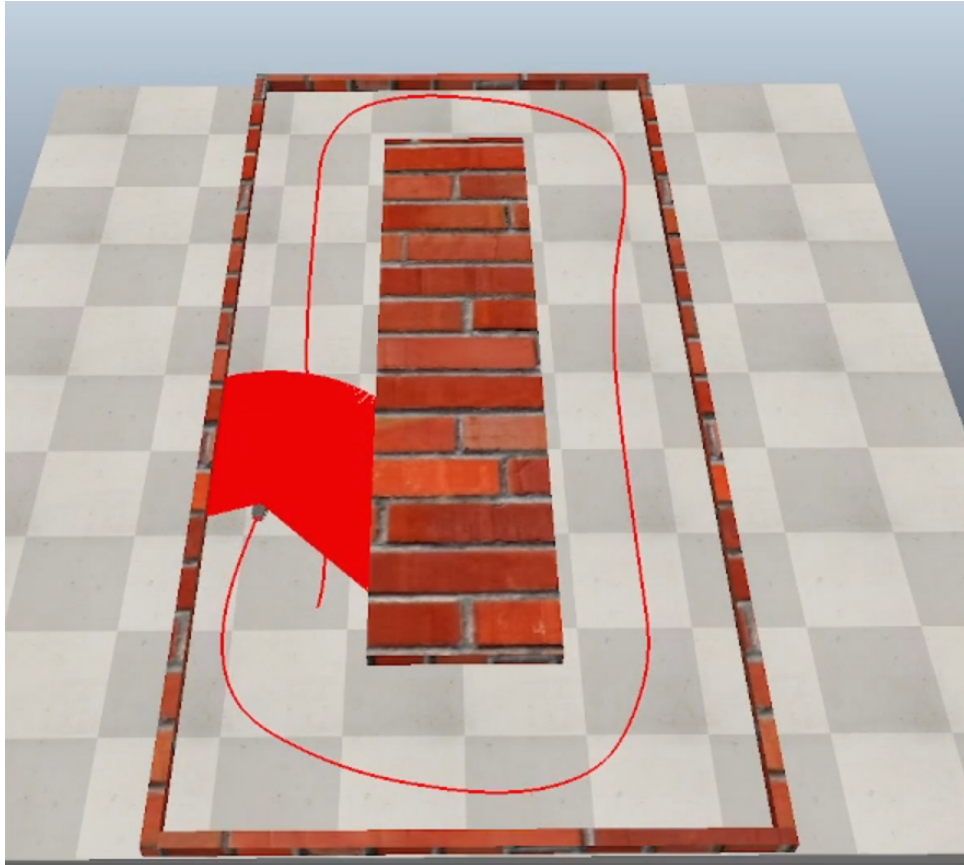


Figura 3.1: Resultado de la navegación con estimación de paredes en Coppelia-Sim: el robot MATLAB sigue el pasillo de forma centrada usando los modelos de pared izquierda y derecha.

En tramos rectos, el comportamiento es notablemente más suave que con el método de ventanas angulares de Lab-MR 1-II porque la regresión integra todas las lecturas del sector y filtra el ruido individual. La orientación del robot se obtiene directamente de las pendientes m_L y m_R , sin necesidad de la heurística de comparar lecturas a $\pm 15^\circ$ que usábamos antes. La velocidad de cruce se ha podido subir a $0,5 \text{ m s}^{-1}$ frente a los $0,25 \text{ m s}^{-1}$ de la práctica anterior, gracias a la mayor estabilidad del estimador.

En las curvas, esta versión no implementa un modo dedicado: cuando una pared sale del sector de detección el intercepto se vuelve infinito y la cláusula de guarda detiene el robot. La integración con el modo curva de Lab-MR 1-II queda como mejora natural pero no la pide el enunciado de esta práctica.

3.4. Comentarios finales

La sustitución de `averageRangeInWindow()` por `extractWall()` cumple lo que pide el enunciado: las paredes se modelan como rectas $y = mx + b$ por mínimos cuadrados, el punto objetivo se calcula analíticamente como el centro del pasillo a la distancia de look-ahead, y el control mantiene Pure Pursuit. La estimación por regresión es más robusta al ruido que el promedio en ventanas estrechas, lo que se traduce en trayectorias más suaves y velocidad de cruce más alta en tramos rectos.

4. Lab-MR 3: Evitar obstáculos mediante campos potenciales

4.1. Objetivo

Completar el script `plantilla_campos_potenciales.m` para que el robot navegue de forma reactiva desde un origen hasta un destino sobre el mapa `mapa1_150.png`, utilizando únicamente las fuerzas de atracción al objetivo y de repulsión de los obstáculos detectados en cada iteración. Realizar varios experimentos con diferentes orígenes y destinos, probar configuraciones de mínimo local, y responder a la pregunta del enunciado: **¿puede el método encontrar la solución cambiando solo α , β o el rango de influencia?**

4.2. Implementación

El código a completar es el bucle de control del robot dentro de la plantilla proporcionada. La estructura de la plantilla (carga del mapa, selección con `ginput`, llamada a `SimulaLidar`, dibujado incremental con `plot/drawnow`) se mantiene tal cual; solo se añaden los cálculos de fuerzas y la actualización de la pose. Las decisiones no obvias son:

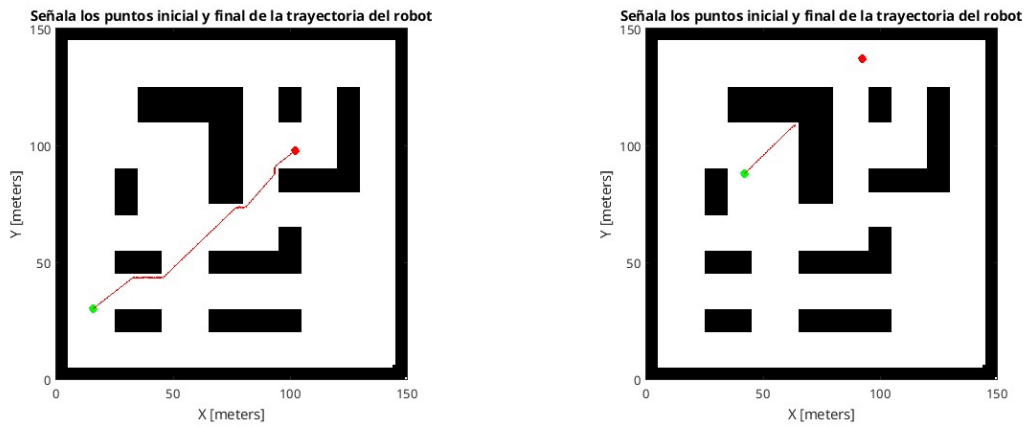
Filtrado de NaN antes de calcular la repulsión. `SimulaLidar` (envoltura de `rayIntersection`) devuelve NaN para los rayos que no impactan dentro del alcance. Si no se descartan con una máscara lógica antes de calcular distancias, las operaciones vectoriales se contaminan y la repulsión pierde sentido.

Recorte por rango de influencia. La fórmula repulsiva del enunciado se anula exactamente en $d = D$, pero evaluarla para todos los rayos del láser es innecesario. Se filtran previamente los obstáculos a $d_i < D$ y solo se itera sobre ellos.

Detección de mínimo local por límite de iteraciones. Se considera que el robot ha quedado atrapado cuando alcanza 1000 iteraciones sin satisfacer la condición de llegada $\|q_{\text{dest}} - q\| < v$. La plantilla ya emite el mensaje correspondiente.

4.3. Pruebas

Sobre el mapa `mapa1_150.png`, con los parámetros por defecto de la plantilla ($\alpha = 1$, $\beta = 100$, $D = 1.5$, paso $v = 0.4$), se han realizado experimentos con orígenes y destinos seleccionados con `ginput`. La Figura ?? muestra los dos comportamientos típicos.



(a) Caso favorable: el robot bordea los obstáculos y alcanza el destino con una trayectoria suave.

(b) Mínimo local: atracción y repulsión se cancelan y el robot queda atrapado tras agotar las 1000 iteraciones.

Figura 4.1: Trayectorias reactivas obtenidas con campos potenciales sobre `mapa1_150.png`. Círculo verde: origen. Círculo rojo: destino. Traza roja: recorrido del robot.

4.3.1. Efecto de los parámetros

Variando α y β se observa el equilibrio entre avance y evitación: con β pequeño el robot se acerca demasiado a las paredes, con β grande la trayectoria se aleja innecesariamente y aparecen oscilaciones. Variando D se controla cuándo empieza a desviarse el robot: D pequeño produce reacciones bruscas en cercanías, D grande adelanta la desviación. Los valores por defecto ofrecen un compromiso adecuado en este mapa.

4.3.2. ¿Pueden los parámetros resolver un mínimo local?

No. Un mínimo local es un punto q^* donde $\mathbf{F}_{\text{atr}}(q^*) + \mathbf{F}_{\text{rep}}(q^*) = \mathbf{0}$. Cambiar α y β *escala* ambas componentes, pero si las fuerzas se cancelan en q^* con una pareja (α, β) , también se cancelan con cualquier $(\lambda\alpha, \lambda\beta)$. Modificar la relación β/α desplaza el punto de cancelación a otra ubicación del entorno pero no elimina el mínimo, simplemente lo traslada. Variar D tampoco rompe la cancelación: cambia qué obstáculos contribuyen a la repulsión, pero en cualquier configuración con simetría (cuello en U, paredes enfrentadas) la cancelación vectorial sigue siendo posible.

Los mínimos locales son por tanto una limitación **estructural** del método reactivo puro, no un problema de calibración. Resolverlos requiere extender el algoritmo (perturbación aleatoria, campo tangencial, información global), vía que se explora en Lab-MR 6.

4.4. Comentarios finales

La implementación cumple lo que pide el enunciado: el robot navega reactivamente, alcanza el destino en configuraciones favorables y queda detectado como atrapado en configuraciones de mínimo local. La respuesta a la pregunta del enunciado es negativa: ajustar parámetros no resuelve los mínimos locales, que requieren modificar el algoritmo. Esa extensión se aborda en Lab-MR 6, donde la planificación global (Dijkstra/A*) actúa como guía y los campos potenciales se complementan con un campo tangencial y una maniobra de escape por estancamiento.

5. Lab-MR 4: Planificación de caminos I (Dijkstra)

5.1. Objetivo

Implementar en MATLAB una función con la signatura

```
[coste, ruta] = dijkstra(G, origen, destino)
```

que devuelva el coste y la secuencia de nodos del camino óptimo entre dos nodos de un grafo dirigido y ponderado, y validarla sobre los dos grafos del enunciado: el grafo de ejemplo de 7 nodos y la matriz J de grafos.mat (25 nodos, dirigida y asimétrica).

El convenio del enunciado es que $G(i, j) = 0$ representa ausencia de arco $i \rightarrow j$ y los pesos son no negativos, condición necesaria para que Dijkstra sea correcto.

5.2. Implementación

La función sigue el esquema clásico de Dijkstra (selección del nodo abierto con menor distancia, relajación de aristas, marcado como cerrado), por lo que aquí solo comentamos las decisiones que no son evidentes del enunciado.

Selección del mínimo por búsqueda lineal. Para los tamaños con los que se trabaja ($N \leq 25$), una búsqueda lineal sobre los nodos abiertos en cada iteración es más clara y rápida que mantener una cola de prioridad. La complejidad $O(N^2)$ resultante es perfectamente aceptable para este problema; con grafos mucho más grandes habría que reconsiderarlo.

Aceptar el struct de load directamente. El enunciado entrega los grafos dentro de grafos.mat. Para evitar el paso intermedio de extraer la matriz a mano, la función acepta tanto una matriz como el struct devuelto por load: si recibe el struct, busca el campo J (la matriz del enunciado) y, en su defecto, la primera matriz cuadrada numérica que encuentre. Así se puede invocar como `dijkstra(load('grafos.mat'), 1, 7)` sin pasos intermedios.

Corte temprano. Como interesa el camino a un destino concreto, el bucle se detiene en cuanto se extrae $u = t$ del conjunto abierto. No cambia la complejidad asintótica, pero reduce el trabajo medio.

Validaciones de entrada. Antes del bucle se comprueba que G es cuadrada, que los costes son finitos y no negativos, y que los índices de origen y destino están dentro del rango. Si $\text{origen} == \text{destino}$ se devuelve $\text{coste} = 0$ sin entrar al bucle. Si la diagonal contiene autolazos, se avisa pero no se aborta: el filtro $G(u, v) > 0$ al recorrer vecinos los ignora automáticamente.

Reconstrucción de la ruta. Si $d[t] = \infty$ al terminar, se devuelve $\text{coste} = \text{Inf}$ y $\text{ruta} = []$ (destino inalcanzable). En caso contrario se reconstruye la ruta retrocediendo por el vector de predecesores hasta el origen.

5.3. Pruebas

5.3.1. Grafo de ejemplo (7 nodos)

Sobre el grafo G de 7 nodos del enunciado:

$$G = \begin{bmatrix} 0 & 2 & 0 & 0 & 9 & 0 & 0 \\ 2 & 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 5 & 0 & 0 & 0 \\ 0 & 2 & 5 & 0 & 1 & 2 & 4 \\ 9 & 0 & 0 & 1 & 0 & 4 & 0 \\ 0 & 0 & 0 & 2 & 4 & 0 & 1 \\ 0 & 0 & 0 & 4 & 0 & 1 & 0 \end{bmatrix}$$

la llamada `dijkstra(G, 1, 7)` devuelve $\text{coste} = 7$ y $\text{ruta} = [1 \ 2 \ 4 \ 6 \ 7]$, que coincide con el resultado esperado del enunciado.

Figura 5.1: Grafo de 7 nodos con la ruta óptima de 1 a 7 resaltada en rojo. Coste 7, ruta [1 2 4 6 7].

5.3.2. Matriz J de grafos.mat (25 nodos)

La matriz J es un grafo dirigido y asimétrico de 25 nodos. La asimetría es relevante: por ejemplo $J(1, 7) \neq J(7, 1)$, lo que hace que las rutas de ida y vuelta entre el mismo par de nodos no sean iguales.

Tabla 5.1: Resultados de `dijkstra` sobre la matriz J (25 nodos, dirigida).

Origen	Destino	Coste	Ruta
1	19	16	[1 7 13 17 23 18 19]
19	1	19	[19 20 15 10 5 9 8 3 2 1]
25	19	11	[25 20 15 14 18 19]
19	25	4	[19 20 25]
1	23	12	[1 7 13 17 23]
23	1	22	[23 18 14 15 10 5 9 8 3 2 1]
1	24	∞	no alcanzable
23	22	40	[23 18 14 15 10 5 9 8 3 2 1 7 13 17 12 11 16 22]

Conviene fijarse en dos casos. Para $1 \rightarrow 23$ el coste es 12 y la ruta usa 5 nodos; para $23 \rightarrow 1$ el coste sube a 22 y la ruta a 10 nodos. Más extremo es $23 \rightarrow 22$: aunque 22 y 23 son nodos próximos en el mapa, los arcos disponibles obligan a recorrer casi todo el grafo (coste 40, 18 nodos). El caso $1 \rightarrow 24$ confirma que la función maneja correctamente la inalcanzabilidad.

También se han probado los casos límite `origen == destino` (devuelve coste 0 y ruta de un solo nodo) y entrada con pesos negativos (rechazada con error en la validación).

5.4. Comentarios finales

La implementación cumple lo que pide el enunciado y se ha validado contra todos los casos propuestos. La búsqueda lineal del mínimo no es la opción más eficiente posible, pero para los tamaños de `grafos.mat` no merece la pena complicarla con una cola de prioridad; en grafos mucho más dispersos y grandes sería el siguiente paso a mirar.

Esta misma función se reutiliza en Lab-MR 5 como referencia para comparar contra A^* , y en Lab-MR 6 como planificador global cuya ruta se entrega luego a la navegación local reactiva.

6. Lab-MR 5: Planificación de caminos II (A*)

6.1. Objetivo

Implementar en MATLAB una función con la signatura

```
[coste, ruta] = aestrella(G, H, origen, destino)
```

que aplique el algoritmo A* sobre un grafo dirigido y ponderado usando una matriz de heurísticas H pasada externamente. Validarla con el grafo de 7 nodos del enunciado, comprobar que la H proporcionada no es admisible para el caso $7 \rightarrow 4$ (el resultado es subóptimo), y proponer una H' admisible que recupere la ruta óptima.

6.2. Implementación

La función sigue el esquema clásico de A* (selección del nodo abierto con menor $f = g + h$, relajación de aristas), con las mismas decisiones de diseño que la implementación de Dijkstra de Lab-MR 4, por lo que aquí solo se comentan las particularidades.

Heurística pasada como matriz externa. La firma del enunciado entrega H como argumento, no se calcula internamente. La función toma $h(v) = H(v, \text{destino})$ en cada relajación. Esto permite estudiar el efecto de heurísticas no admisibles sin tocar la función.

Selección del mínimo por búsqueda lineal. Igual que en Dijkstra, se usa una búsqueda lineal sobre los nodos abiertos. La complejidad $O(N^2)$ resultante es aceptable para los tamaños del enunciado.

Corte temprano. El bucle se detiene al extraer el destino del conjunto abierto. Bajo heurística admisible esto es seguro; bajo heurística no admisible puede devolver una ruta subóptima — precisamente lo que se ilustra en el caso $7 \rightarrow 4$.

Validaciones de entrada. Se comprueba que G y H son cuadradas $N \times N$ del mismo tamaño, con costes y heurísticas finitos y no negativos, y que origen

y destino están en rango. El caso `origen == destino` devuelve `coste = 0` sin entrar al bucle.

6.3. Pruebas

Sobre el grafo G y la matriz heurística H del enunciado:

$$G = \begin{bmatrix} 0 & 2 & 0 & 0 & 9 & 0 & 0 \\ 2 & 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 5 & 0 & 0 & 0 \\ 0 & 2 & 5 & 0 & 1 & 2 & 4 \\ 9 & 0 & 0 & 1 & 0 & 4 & 0 \\ 0 & 0 & 0 & 2 & 4 & 0 & 1 \\ 0 & 0 & 0 & 4 & 0 & 1 & 0 \end{bmatrix} \quad H = \begin{bmatrix} 0 & 2 & 5 & 5 & 5 & 6 & 9 \\ 2 & 0 & 2 & 3 & 6 & 6 & 8 \\ 5 & 2 & 0 & 2 & 6 & 6 & 7 \\ 5 & 3 & 2 & 0 & 5 & 4 & 6 \\ 5 & 6 & 6 & 5 & 0 & 1 & 5 \\ 6 & 6 & 6 & 4 & 1 & 0 & 1 \\ 9 & 8 & 7 & 6 & 5 & 1 & 0 \end{bmatrix}$$

6.3.1. Caso $1 \rightarrow 7$ con H original

La llamada `aestrella(G, H, 1, 7)` devuelve `coste = 7` y `ruta = [1 2 4 6 7]`, óptimo y coincidente con el resultado de Dijkstra.

6.3.2. Caso $7 \rightarrow 4$ con H original (heurística no admisible)

La llamada `aestrella(G, H, 7, 4)` devuelve `coste = 4` y `ruta = [7 4]`, que **no** es óptima: la ruta `[7 6 4]` tiene coste 3.

El motivo es que $H(7,4) = 6$ y $H(6,4) = 4$ sobreestiman los costes reales $h^*(7,4) = 3$ y $h^*(6,4) = 2$. Trazando el bucle: tras expandir el origen 7, los abiertos son 4 con $f(4) = 4 + H(4,4) = 4$ y 6 con $f(6) = 1 + H(6,4) = 5$. A^* extrae el de menor f (que es el destino 4), aplica corte temprano y devuelve `[7, 4]` sin llegar a expandir 6. La condición $h(n) \leq h^*(n)$ falla, por lo que la garantía de optimalidad de A^* no aplica.

6.3.3. Heurística admisible propuesta

Una heurística admisible para el destino 4 debe cumplir $H'(i, 4) \leq h^*(i, 4)$ para todo i . Los costes reales mínimos a 4 (calculados con Dijkstra sobre G) son:

$$h^*(:, 4) = [4 \ 2 \ 3 \ 0 \ 1 \ 2 \ 3]^T$$

Sustituyendo la columna 4 de H por estos valores se obtiene la matriz H' . Con esa heurística, `aestrella(G, H', 7, 4)` devuelve `coste = 3` y `ruta = [7 6 4]`: A^* recupera el óptimo porque ahora $f(6) = 1 + 2 = 3 < f(4) = 4$, y explora primero 6.

6.4. Comentarios finales

La implementación cumple lo que pide el enunciado y los tres casos confirman el papel crítico de la admisibilidad: con heurística admisible A^* devuelve la ruta óptima, y sin admisibilidad puede devolver rutas subóptimas pese a ser un algoritmo informado.

Esta misma función se reutiliza en Lab-MR 6 como planificador global, donde se construye una heurística euclídea consistente recalculando una matriz de costes basada también en distancias euclídeas (necesario para que la heurística sea consistente, no solo admisible).

7. Lab-MR 6: Navegación completa

7.1. Objetivo

Implementar un programa de MATLAB que combine la planificación global (Dijkstra de Lab-MR 4, y posteriormente A*) con la navegación local reactiva por campos potenciales (Lab-MR 3) sobre el mapa `mapa2.pgm` y el grafo `mapa2.m`. El programa pide los nodos de inicio y destino, representa gráficamente el mapa con la trayectoria del robot e indica si se ha alcanzado el destino. El enunciado pide además probar la trayectoria desde el nodo 1 hacia la zona superior derecha y atravesar la región entre los nodos 24 y 32, identificar el problema que aparece en esa zona, proponer una mejora, y sustituir Dijkstra por A* con una heurística euclídea consistente.

7.2. Implementación

La solución se distribuye en dos scripts principales, uno por planificador (`navegacion_autonoma_p7_` y `navegacion_autonoma_p7_Aestrella.m`), que comparten la misma navegación local. Las funciones de campo potencial están encapsuladas en `fuerza_atractiva.m` y `fuerza_repulsiva.m`, y los algoritmos de planificación en `dijkstra.m` y `Aestrella.m`. El mapa y el grafo se cargan con `imread('mapa2.pgm')` y ejecutando `mapa2.m`, y se visualizan con `imshow(flipud(img))` y `set(gca, 'YDir', 'normal')` para mantener la orientación del enunciado (eje y creciente hacia arriba).

Campos potenciales. El campo atractivo es lineal con la distancia, $\mathbf{F}_{\text{att}} = K_{\text{att}} (p_{\text{obj}} - p)$ con $K_{\text{att}} = 2$. La fuerza repulsiva se calcula recorriendo una ventana cuadrada de lado $2d_0 + 1$ alrededor del robot ($d_0 = 6$ píxeles); por cada píxel obstáculo se acumula una contribución proporcional a $K_{\text{rep}} \left(\frac{1}{d} - \frac{1}{d_0} \right) \frac{1}{d^2}$ en la dirección al obstáculo, con $K_{\text{rep}} = 20$. La consulta de obstáculos se hace directamente sobre la imagen original mediante `img(y,x)==0`, en coincidencia con la convención del enunciado.

Mejora anti-mínimos locales. Para combatir los mínimos locales identificados en el apartado 2 del enunciado, la fuerza resultante se construye como

$$\mathbf{F}_{\text{total}} = 3 \mathbf{F}_{\text{att}} + \mathbf{F}_{\text{rep}},$$

priorizando la atracción al subobjetivo frente a la repulsión. Además, si la resultante tiene componente negativa en la dirección al objetivo ($\mathbf{F}_{\text{total}} \cdot (p_{\text{obj}} - p) < 0$), se sustituye por $\mathbf{F}_{\text{att}}$ pura. Esta regla evita que el robot retroceda o quede atrapado en equilibrios estables entre atracción y repulsión, y es la única diferencia frente al esquema básico de Lab-MR 3.

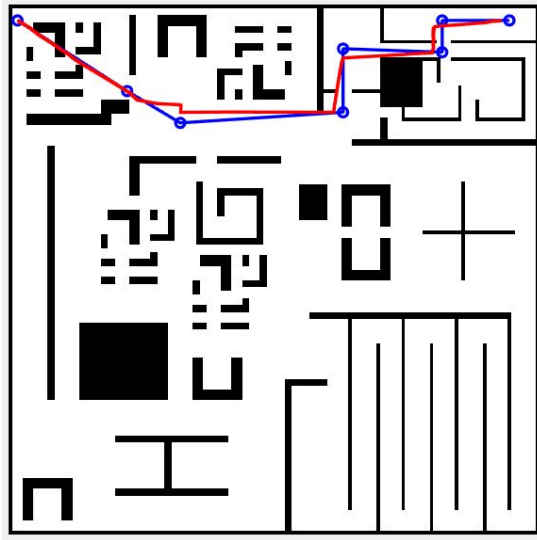
A* con heurística consistente. A* solo es óptimo cuando los pesos del grafo coinciden con la métrica usada en la heurística. Antes de invocar Aestrella, se construye una matriz de costes C^{eucl} que mantiene la adyacencia del grafo original pero sustituye cada peso $C(i, j)$ por $\|p_i - p_j\|_2$. La heurística utilizada es $H(i, j) = \|p_i - p_j\|_2$, euclídea entre cualquier par de nodos. Con esta construcción, la heurística es consistente por definición.

7.3. Pruebas

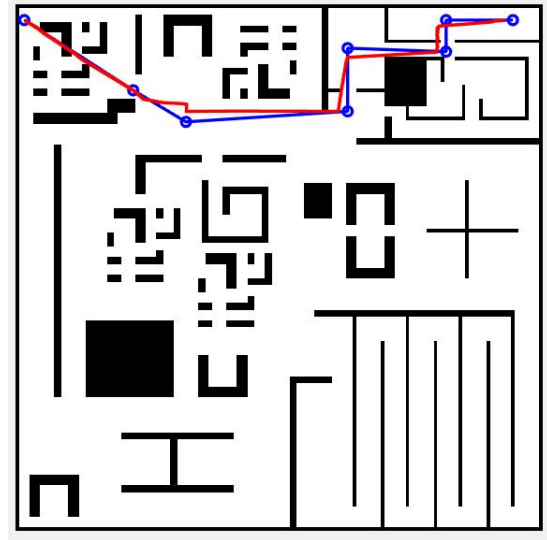
Se prueba el sistema en los dos escenarios solicitados (1→12 en la zona superior derecha del mapa y 24→32 en el pasillo inferior derecho), con ambos planificadores y la mejora anti-mínimos locales activa. En los cuatro casos el robot completa la ruta.

7.3.1. Trayectoria del nodo 1 a la zona superior derecha (nodo 12)

La ruta planificada atraviesa varias *habitaciones* con pasillos estrechos. Tanto Dijkstra como A* producen rutas globales equivalentes en coste euclídeo, y la trayectoria real del robot (rojo) se cinea al camino planificado (azul) sin estancamientos.



(a) Planificador: Dijkstra.

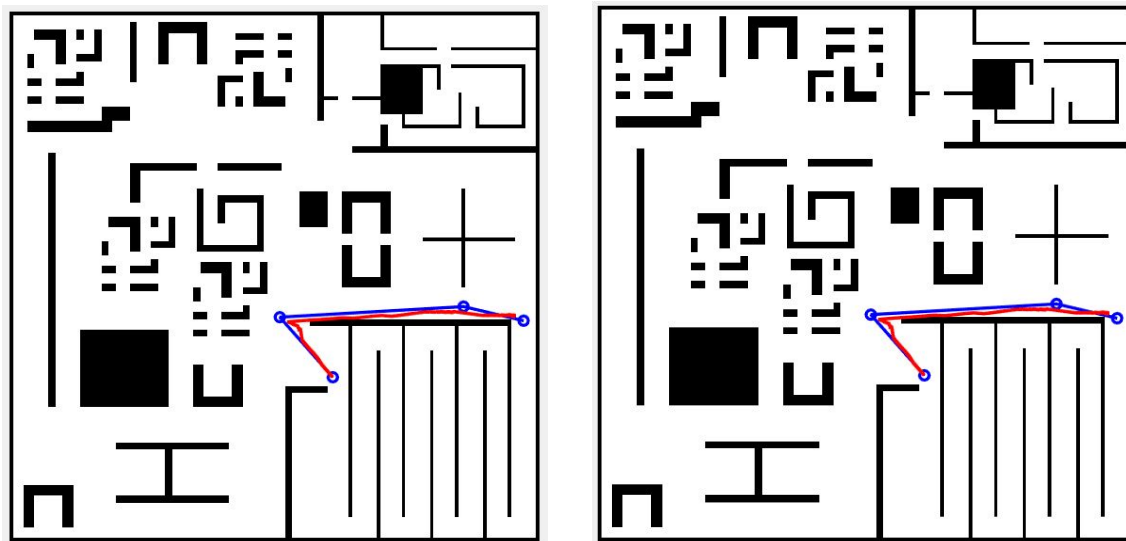


(b) Planificador: A*.

Figura 7.1: Trayectoria 1→12 con ambos planificadores. Ruta global en azul, trayectoria real del robot en rojo.

7.3.2. Zona conflictiva entre nodos 24 y 32

Esta región contiene un pasillo estrecho con obstáculos dispuestos simétricamente respecto al eje del robot. En el esquema básico de Lab-MR 3 ($F_{total} = F_{att} + F_{rep}$), atracción y repulsión se cancelan en la entrada del pasillo y el robot queda atrapado en un mínimo local clásico. Con la mejora descrita en el apartado anterior (atracción con factor $\times 3$ y *fallback* al atractivo cuando la resultante apunta lejos del objetivo) el robot supera la zona y alcanza el destino con ambos planificadores.



(a) Planificador: Dijkstra.

(b) Planificador: A*.

Figura 7.2: Trayectoria 24→32 a través de la zona conflictiva, con la mejora anti-mínimos locales activada.

7.4. Comentarios finales

La integración jerárquica de planificación global y control local funciona como pide el enunciado: la ruta del planificador (Dijkstra o A*) define la secuencia de subobjetivos, y los campos potenciales ejecutan cada tramo evitando obstáculos. El problema de mínimo local en la zona 24–32 se resuelve con dos modificaciones sencillas (factor $\times 3$ sobre la atracción y *fallback* al atractivo puro cuando la resultante retrocede). El reemplazo de Dijkstra por A* funciona al construir una matriz de costes euclídea coherente con la heurística euclídea, condición necesaria para mantener la garantía de optimalidad de A*.